DEPARTMENT OF INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Thesis type (Bachelor's Thesis in Informatics, Master's Thesis in Informatics: Games Engineering, ...)

Thesis title

Author

DEPARTMENT OF INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Thesis type (Bachelor's Thesis in Informatics, Master's Thesis in Informatics: Games Engineering, ...)

Thesis title

Titel der Abschlussarbeit

Author:	Author
Supervisor:	Supervisor
Advisor:	Advisor
Submission Date:	Submission date

I confirm that this thesis type (bachelor's thesis in informatics, master's thesis in informatics: games engineering, ...) is my own work and I have documented all sources and material used.

Munich, Submission date Author

Acknowledgments

Abstract

Contents

Acknowledgments **iii**

Abstract **iv**

1. Introduction **1**
1.1. Section 1
1.1.1. Subsection 1

2. Methods and Technology **4**

3. Implementation **5**
3.1. Project Setup 5
3.1.1. Source Code Organization 5
3.2. Move Semantics 6
3.3. Shared Pointers 7
3.4. The Builder Pattern 7

A. General Addenda **8**
A.1. Detailed Addition 8

B. Figures **9**
B.1. Example 1 9
B.2. Example 2 9

List of Figures **10**

List of Tables **11**

Bibliography **12**

1. Introduction

Use with pdfLaTeX and Biber.

1.1. Section

Citation test (with Biber) [1].

1.1.1. Subsection

See Table 1.1, Figure 1.1, Figure 1.2, Figure 1.3, Figure 1.4, Figure 1.5.

Table 1.1.: An example for a simple table.

A	B	C	D
1	2	1	2
2	3	2	3

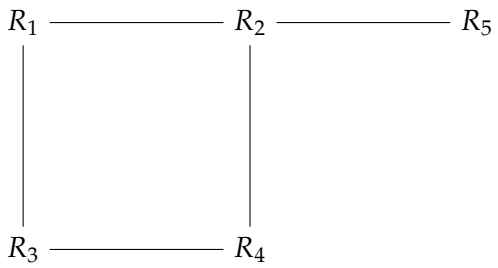

Figure 1.1.: An example for a simple drawing.

This is how the glossary will be used.

Donor dye, ex. Alexa 488 (D_{dye}), Förster distance, Förster distance (R_0), and k_{DEAC}. Also, the TUM has many computers, not only one Computer. Subsequent acronym usage will only print the short version of Technical University of Munich (TUM) (take care of plural, if needed!), like here with TUM, too. It can also be –> hidden[1] <–.

[(TODO: Now it is your turn to write your thesis.
This will be a few tough weeks.)]
[(DONE: NEVERTHELESS, CELEBRATE IT WHEN IT IS DONE!)]

[1] Example for a hidden TUM glossary entry.

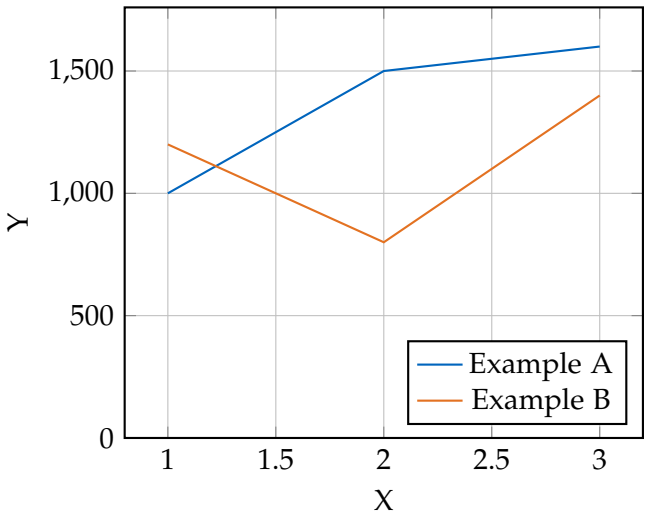

Figure 1.2.: An example for a simple plot.

```
SELECT * FROM tbl WHERE tbl.str = "str"
```

Figure 1.3.: An example for a source code listing.

Figure 1.4.: Includegraphics searches for the filename without extension first in logos, then in figures.

Figure 1.5.: For pictures with the same name, the direct folder needs to be chosen.

(a) The logo.

(b) The famous slide.

Figure 1.6.: Two TUM pictures side by side.

2. Methods and Technology

Use with pdfLaTeX and Biber.

3. Implementation

3.1. Project Setup

The projects as well as this thesis' source code are avaibable in the following Github repository:
`https://github.com/mulsicpp/bachelor_thesis_2025.git`
The project is organized in the following directories:

- "assets": This folder contains the projects' assets. It is further divided into scenes, which contains the glTF test scenes, and "shaders", which contains the shaders, that are used in the different pipelines.
- "external": This folder contains all of the "complex""" external libraries. "Complex" in this context means, that the library has multiple header or source files or it is in binary form.
- "premake": This folder contains the premake executables for both windows and linux.
- "src": This folder contains all of our source files for the project. It is described in more detail in the next chapter.

Additionally, there are some scripts, that are used to build the project.

3.1.1. Source Code Organization

In this section we will specifically look at how the "src" folder is organised.

Directly in the "src" folder we find "main.cpp", which contains our entry point of the application. The "external" folder once again contains external libraries, but this time the libraries, which are in the form of a single header file. These usually need to be included in a C/C++ file and need to be compiled. That's why it makes sense to keep these libraries with our other source files. The "utils" folder contains utilities, that may be useful all throughout the project. It includes functionalities like debug logging, move semantics and file loading. All the folders with a "vk_" prefix contain code, which abstracts the low-level Vulkan API code. The goal of these abstractions is, that the low-level code should only ever be used in these folders. The rest of the project should only use the abstracted API. Each of the folders focuses on a different area of Vulkan:

- "vk_core": The most important class in this folder is the `Context` class. It is designed as a singleton, which contains the `VkInstance`, `VkPhysicalDevice`, `VkDevice`, `VkQueues` and `VmaAllocator`. After the context gets created it can be accessed anywhere. The

folder also contains the `Handle<T>` class, which ensures that a handle only has one owner, the `CommandBuffer` abstraction and functionality to deals with `VkFormat`.

- "vk_resources": This folder contains Vulkans resource objects like `Buffer` and `Image`, as well as `ImageView` and `SubBuffer`.
- "vk_pipeline": This folder contains Vulkan objects, that are related to the setup of a pipeline. This includes `Pipeline`, `PipelineLayout`, `Shader`, `RenderPass`, `Framebuffer` and `DescriptorPool`.
- "vk_sync": Contains the vulkan synchronization objects `Fence` and `Semaphore`.
- "scene": This folder contains the `Scene` class, which describes a dynamic scene with a scene graph. All the other classes used by or contained in the scene, like `Mesh`, `Node`, `Camera` and `Animation`, can also be found in this folder.
- "rendering": This folder contains 3 different renderers:
  - `Rasterizer`: Render a scene from the viewpoint of a camera into a framebuffer using traditional rasterisation.
  - `Raytracer`: Render a scene from the viewpoint of a camera into an image using modern hardware-accelerated raytracing.
  - `Skinner`: Calculates the skinned positions of a mesh during an animation.

3.2. Move Semantics

As already mentioned in the Methods and Technology chapter, Rust was originally considered as programming language, because it is modern and memory safe. One of Rust's core features is ownership, which means that each value/object can only be owned and managed by only one variable, and when that variable goes out of scope, it alone is responsible for cleaning up the object. When that variable gets assigned to another variable, the new variable becomes the new owner (i.e. the value gets moved). Ownership would be a very nice feature to manage Vulkan objects. Vulkan objects should not be copied on assignment but only moved and when the object is no longer needed (i.e. the owner goes out of scope), it should automatically be destroyed. While C++ does not support ownership directly, it does have so-called move semantics, with which we can emulate ownership. The ownership of Vulkan handles is managed by the `Handle<T>` class. To do this the class declares a default constructor, move constructor and destructor and overrides the move assignment operator. It is also important, that the copy constructor and copy assignment operator are deleted. The reason for that is that copying a Vulkan object is usually very expensive or not even possible and it is almost never necessary. When we want to copy an object, we need to explicitly create a new one first. This avoids unnecessary and complicated accidental creation and copying of new objects.

3.3. Shared Pointers

Since C++11 the language supports so-called smart pointers, which are supposed to make memory management safer and less complicated. One of these smart pointers is `std::shared_ptr<T>` (or our alias `ptr::Shared<T>`). This type of pointer allocates an object, which can then be shared between multiple owners. The object lives while it has at least one owner, and the last object to go out of scope cleans up the object. This structure can be combined very nicely with our previously defined handle. A Vulkan object can only have one owner, which is a harsh limitation and often causes problems. If we wrap the object in a shared pointer, it can be shared across multiple owners. We also added the class `ToShared<T>`, which a class can inherit from. An instance of that class can then be moved into a shared pointer by calling `to_shared()`.

3.4. The Builder Pattern

As mentioned, before we want the creation of Vulkan objects to be very explicit. The first method to create objects in C++ you think of are constructors, but they prove to be problematic for a couple of reasons. The first problem is that constructors are often called implicitly (e.g. the default constructor gets called when declaring a variable). Default construction only makes sense for very few Vulkan objects (like for example a VkFence) and we do not want a Vulkan object to be created when implicitly (or explicitly) calling the default constructor. So, for that reason, the default constructor simply does not create a Vulkan object or, in other words, leaves the handle at `VK_NULL_HANDLE`. Now if a different constructor actually creates an object, the behavior of constructors becomes inconsistent regarding Vulkan object creation. In addition to that, Vulkan objects are almost always created using a `*CreateInfo` struct, which may take a lot of parameters. To solve all of these issues we implemented a `Builder` class for most Vulkan objects. Some exceptions are `Fence` and `ImageView`, which are created with `Fence::create(...)` and `ImageView::create_from(...)` respectively. This makes it very clear when a Vulkan object is created, since it can only be created using a `Builder` or a static method (like `Fence::create(...)`) and especially it can never be created by a constructor. This is also the reason Vulkan objects never have constructor, besides the default constructor.

```
vk::Buffer buffer; // declare a buffer (handle is VK_NULL_HANDLE)

buffer = vk::BufferBuilder()
    .add_usage(VK_BUFFER_USAGE_TRANSFER_DST_BIT)
    .add_usage(VK_BUFFER_USAGE_VERTEX_BUFFER_BIT)
    .memory_usage(VMA_MEMORY_USAGE_GPU_ONLY)
    .size(128)
    .build();
```

A. General Addenda

If there are several additions you want to add, but they do not fit into the thesis itself, they belong here.

A.1. Detailed Addition

Even sections are possible, but usually only used for several elements in, e.g. tables, images, etc.

B. Figures

B.1. Example 1

✓

B.2. Example 2

✗

List of Figures

1.1. Example drawing 1
1.2. Example plot 2
1.3. Example listing 2
1.4. Something else can be written here for listing this, otherwise the caption will be written! 2
1.5. For pictures with the same name, the direct folder needs to be chosen. 3
1.6. Two TUM pictures side by side. 3

List of Tables

1.1. Example table 1

Bibliography

[1] L. Lamport. *LaTeX : A Documentation Preparation System User's Guide and Reference Manual*. Addison-Wesley Professional, 1994.